

Репликация

Эволюция базы данных:
от одной таблицы до распределенной системы

Репликация

Репликация — это процесс синхронного или асинхронного копирования данных с одного узла (сервера) системы на другие узлы в режиме реального времени.

Это фундаментальный механизм распределенных систем и баз данных, предназначенный для обеспечения отказоустойчивости, высокой доступности и горизонтального масштабирования.

Зачем это нужно? На это есть три главные причины:

1. **Отказоустойчивость (Fault Tolerance):** Если главный сервер "упал", система автоматически переключается на реплику, и сайт продолжает работать.
2. **Масштабирование чтения (Read Scaling):** Миллионы пользователей могут одновременно читать посты, смотреть товары или листать ленту, не перегружая главный сервер.
3. **Географическая близость (Geographic Proximity):** Реплику базы данных можно поставить в дата-центр на другом континенте, чтобы пользователи из этого региона получали данные мгновенно, без задержек на передачу через океан.

Как устроен процесс на техническом уровне

Когда приложение выполняет операцию записи (например, INSERT или UPDATE), в СУБД запускается следующий цепочечный процесс:

1. **Запись в лог (WAL/Binlog):** Сервер **Primary** принимает запрос, изменяет данные в памяти и обязательно фиксирует это изменение в специальном файле — логе транзакций (в PostgreSQL это *Write-Ahead Log (WAL)*, в MySQL — *Binary Log*).
2. **Передача изменений:** Специальный фоновый процесс-демон (отправитель) считывает новые записи из этого лога и пересылает их по сети на серверы-**Replica**.
3. **Применение изменений:** Процесс-приемник на стороне реплики получает бинарные или логические команды, записывает их в свой лог и последовательно выполняет их на своей копии базы данных.

Архитектурные преимущества репликации

- **Горизонтальное масштабирование (Read-Heavy нагрузки):**
Позволяет разгрузить центральный процессор и дисковую подсистему основного сервера, распределяя аналитические и поисковые SELECT-запросы по пулу реплик.
- **Отказоустойчивость (High Availability):**
При выходе из строя узла Primary, оркестратор инициирует процедуру **Failover** — выбирает наиболее актуальную реплику и переводит ее в режим работы Primary.
- **Изоляция инфраструктуры:**
Позволяет снимать резервные копии (dump) с реплики, исключая деградацию производительности «живой» базы данных, с которой работают пользователи.

Failover (аварийное переключение) — это процесс автоматического переключения работы на резервный сервер, компонент или систему, когда основной элемент выходит из строя или начинает работать с ошибками

Как это работает

- **Мониторинг:** Основной и резервный серверы постоянно проверяют состояние друг друга с помощью специальных сигналов (heartbeat).
- **Сбой:** Если основной сервер перестает отвечать, система фиксирует аварию.
- **Переключение:** Нагрузка, трафик или базы данных мгновенно перенаправляются на заранее подготовленную резервную копию.
- **Работа:** Сервис продолжает работать без участия человека (администратора)

Типы репликации

1. По времени подтверждения транзакции:

- **Асинхронная (Asynchronous):**
 - **Процесс:** Primary записывает данные на диск, сразу сообщает приложению «Готово!» и только после этого в фоновом режиме отправляет изменения на реплики.
 - **Специфика:** Самая быстрая. Но есть риск **Replication Lag** (задержки) и потери данных, если Primary «умрет» до того, как успеет отправить лог на реплику.
- **Синхронная (Synchronous):**
 - **Процесс:** Primary принимает запись, отправляет её на реплику и **блокирует** ответ пользователю. Только когда реплика запишет данные к себе и пришлет подтверждение, Primary отвечает приложению об успехе.
 - **Специфика:** Гарантирует сохранность данных (Zero Data Loss), но сильно увеличивает время ответа (Latency) и зависит от скорости сети.
- **Полусинхронная (Semi-synchronous):**
 - **Процесс:** Компромиссный вариант. Primary ждет подтверждения хотя бы от **одной** реплики из всего пула (или ждет, пока реплика просто получит лог в память, без записи на диск). Как только одна реплика ответила — транзакция завершается.

Типы репликации

2. По формату и уровню передаваемых данных:

- **Физическая репликация (Physical / Binary / Постраничная):**
 - **Суть:** Копируются низкоуровневые бинарные изменения файлов базы данных (изменения на уровне байт и страниц диска). Реплика просто в точности повторяет структуру файлов Primary.
 - **Плюсы:** Работает очень быстро, создает минимальную нагрузку на процессор.
 - **Минусы:** Требуется абсолютной идентичности серверов (одинаковая ОС, архитектура процессора и версия СУБД). Реплика, как правило, доступна только на чтение (Read-Only).
- **Логическая репликация (Logical / Построчная):**
 - **Суть:** Передаются логические команды изменений данных (например: «В таблице *users* в строке с *id=42* изменить поле *status* на 'active'»).
 - **Плюсы:** Позволяет реплицировать данные между разными версиями СУБД, разными ОС или даже реплицировать только часть таблиц (выборочная репликация).
 - **Минусы:** Медленнее, так как реплика тратит ресурсы процессора на повторный парсинг и выполнение этих логических инструкций.

Типы репликации

3. По топологии и распределению ролей

- **Master-Slave (Primary-Replica / Однонаправленная):**
 - **Суть:** Есть строго один главный сервер (Primary), который принимает и запись, и чтение. Все остальные серверы (Replica) работают только на чтение и получают изменения исключительно от Primary.
 - **Применение:** Самый популярный паттерн для веб-приложений с высокой нагрузкой на чтение (Read-Heavy).
- **Master-Master (Multi-Master / Разделенная запись):**
 - **Суть:** В системе есть несколько равноправных серверов, и приложение может отправлять запросы на запись (INSERT/UPDATE) на *любой* из них. Изменения синхронизируются между всеми узлами.
 - **Проблема:** Возникают конфликты репликации (когда на двух серверах одновременно изменили одну и ту же строку). Для их решения используются сложные алгоритмы.
- **Peer-to-Peer (Децентрализованная):**
 - **Суть:** Все узлы абсолютно равны, центрального лидера нет. Запись происходит на любой узел, а данные распространяются по сети по принципу «сарафанного радио» (Gossip-протоколы). Часто применяется в NoSQL базах (Cassandra, DynamoDB).

Read Scaling

Масштабирование чтения (Read Scaling) через архитектуру **Primary-Replica** (Master-Slave) — это паттерн проектирования баз данных, при котором нагрузка разделяется между несколькими серверами для повышения производительности и отказоустойчивости.

В этой схеме выделяют два типа серверов:

- **Primary (Master)** — главный сервер. Он принимает все запросы на изменение данных (INSERT, UPDATE, DELETE).
- **Replica (Slave)** — подчиненные серверы. Они копируют данные с Primary и обслуживают запросы на чтение (SELECT).

Как это работает?

- **Запись данных:** Приложение отправляет все операции записи на **Primary**.
- **Репликация:** Primary записывает изменения в свой лог (например, WAL в PostgreSQL) и передает эти изменения на **Replica**.
- **Чтение данных:** Приложение направляет запросы на чтение на одну или несколько реплик. Для распределения этой нагрузки обычно используется балансировщик (Load Balancer).

Преимущества:

- **Высокая производительность на чтение:** Вы можете добавлять новые реплики по мере роста базы пользователей.
- **Отказоустойчивость:** Если одна реплика выйдет из строя, остальные продолжат работу. Если упадет Primary, одну из реплик можно «повысить» до главного сервера.
- **Изоляция тяжелых задач:** На отдельную реплику можно вынести построение аналитических отчетов или резервное копирование (Backup), чтобы они не тормозили основное приложение.

Сложности и недостатки:

- **Задержка репликации (Replication Lag):** Из-за асинхронности пользователь может обновить профиль (запись на Primary) и сразу же не увидеть изменений на странице (чтение из реплики, куда данные еще не долетели). Этот эффект называется *Eventual Consistency* (согласованность в конечном счете).
- **Усложнение логики приложения:** Код приложения должен четко понимать, на какой роутер отправлять запрос — для записи или для чтения.

Read Scaling

В системах масштабирования чтения чаще всего используют **асинхронную репликацию**, так как она обеспечивает максимальную скорость работы.

Чтобы внедрить Primary-Replica, разработчики обычно используют следующие инструменты:

- **На уровне БД:** Встроенные механизмы репликации в PostgreSQL, MySQL, Redis или MongoDB.
- **На уровне инфраструктуры:** Прокси-серверы и балансировщики (например, *ProxySQL* для MySQL или *PgBouncer* / *HAProxy* для PostgreSQL), которые сами распределяют SELECT и WRITE запросы.
- **На уровне кода:** Подключение библиотек (ORM), которые поддерживают концепцию *Database Routing*.

CAP-теорема

CAP-теорема — это фундаментальный закон распределенных систем, который утверждает, что в любой распределенной системе (например, в кластере баз данных) возможно одновременно обеспечить только два из трех следующих свойств:

- **C (Consistency — Согласованность):** Во всех узлах кластера в один и тот же момент времени находятся **одинаковые данные**. Если мы записали новое значение на один сервер, при чтении с любого другого сервера мы гарантированно получим это новое значение или ошибку, но точно не старые данные.
- **A (Availability — Доступность):** Любой работающий узел системы на каждый запрос **всегда возвращает успешный ответ** (не ошибку и не таймаут), но при этом не гарантируется, что данные в ответе будут самыми свежими.
- **P (Partition Tolerance — Устойчивость к разделению):** Система **продолжает работать**, даже если связь между её узлами полностью пропала (сеть разделилась на изолированные части, произошел «Network Split»).

WAL

WAL (Write-Ahead Logging / Журнал упреждающей записи) — это фундаментальный механизм в базах данных (включая PostgreSQL), который гарантирует сохранность данных и их целостность даже в случае внезапного сбоя питания или падения сервера.

Основной принцип WAL формулируется очень просто: **любое изменение данных (добавление, удаление или обновление строки) должно быть зафиксировано в журнале на диске до того, как эти изменения будут применены к самим файлам таблиц**

Какую проблему решает WAL?

В оперативной памяти сервера СУБД работать с данными очень быстро. Но RAM энергозависима: если выключить свет, всё сотрется. Записывать каждое микро-изменение сразу в файлы таблиц на жесткий диск — катастрофически медленно, так как таблицы разбросаны по разным частям диска (это называется произвольной записью / Random I/O).

Решение с WAL:

1. Когда пользователь делает UPDATE, СУБД меняет данные в оперативной памяти (в кэше).
2. Чтобы данные не пропали при сбое, СУБД в ту же миллисекунду дописывает строчку с описанием этой операции (например: «*В блоке №5 изменить значение А на Б*») в конец файла **WAL**.
3. Запись в WAL происходит **последовательно (Sequential I/O)**. Для современных дисков это самая быстрая операция.
4. Как только строчка в WAL сохранена на диск, СУБД уверенно говорит пользователю: «*Транзакция успешно завершена (COMMIT)*».
5. Сами файлы таблиц на диске обновятся позже (в фоновом режиме специальным процессом Checkpointer).

Как WAL связан с репликацией?

- В архитектуре **Primary-Replica** сервер Primary не отправляет реплике SQL-запросы вида UPDATE users.... Вместо этого он **стримит поток байт своего WAL-лога** по сети. Реплика принимает этот бинарный поток, записывает к себе в WAL и непрерывно находится в режиме «вечного восстановления» (Recovery Mode) — она просто бесконечно проигрывает WAL-записи, полученные от Primary, становясь его точной физической копией.